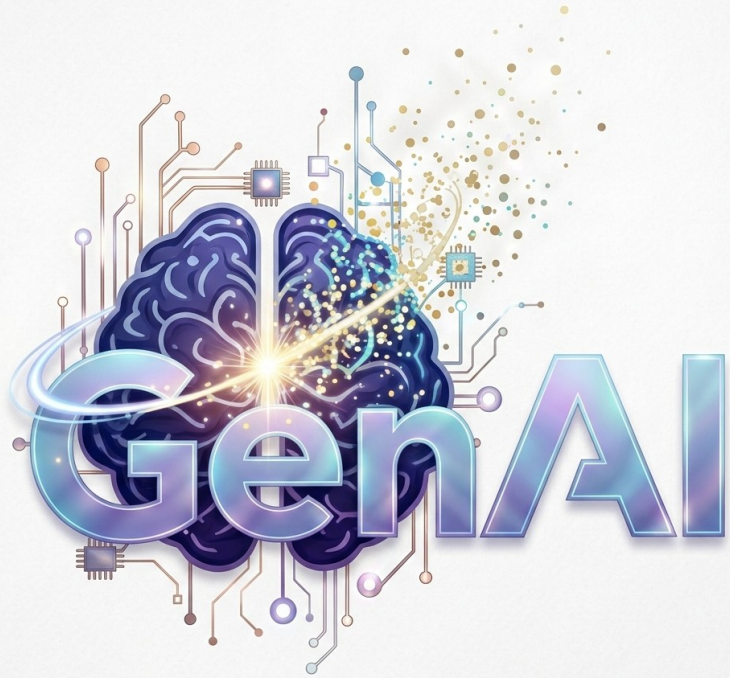
Politecnico di Torino

Generative Artificial Intelligence for graphics and multimedia



Latent diffusion models

Prof. Lia Morra

Dai, Xiaoliang, et al. "Emu: Enhancing image generation models using photogenic needles in a haystack." *arXiv preprint arXiv:2309.15807* (2023).

~~Latent~~ diffusion models

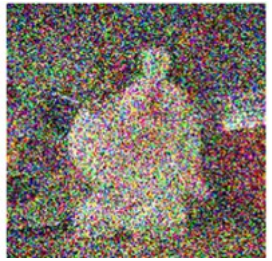
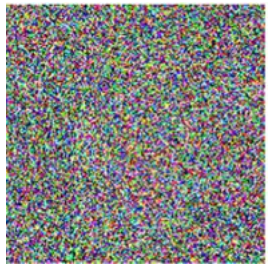
Deep Denoising Probabilistic Model (DDPM) are the first step towards diffusion models:

- **forward diffusion** gradually perturbs the input data.
- **reverse denoising** process that learns to generate data through iterative denoising

Forward process

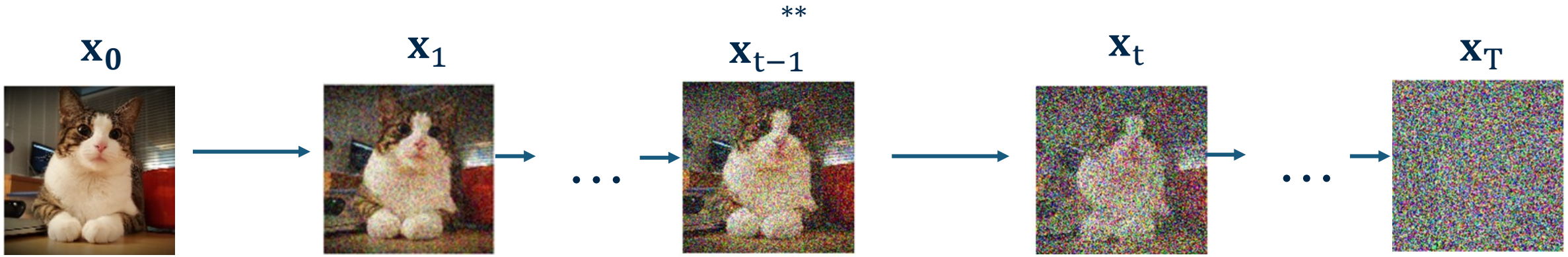
$\mathbf{x_0}$ $\mathbf{x_1}$ $\mathbf{x_{t-1}}$ $\mathbf{x_t}$ $\mathbf{x_L}$



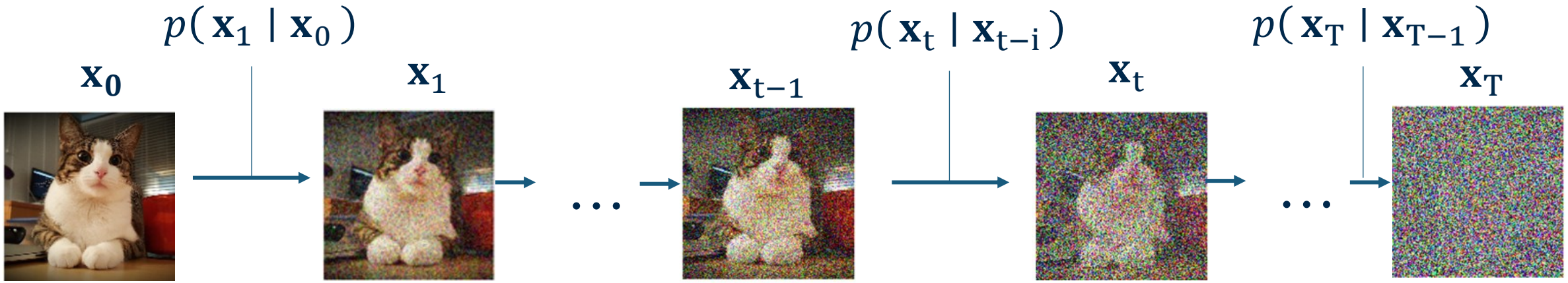
Forward process

In the forward process we are gradually adding Gaussian noise over T* steps

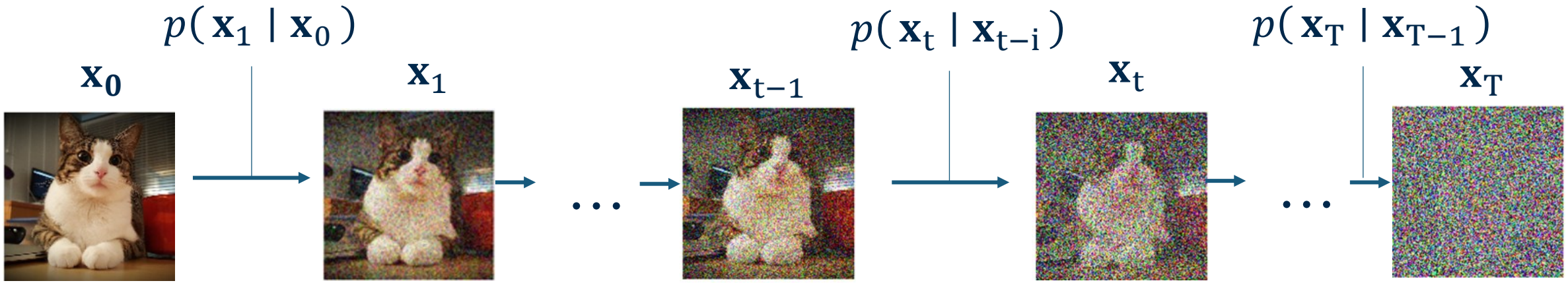


*L is also used in notation
***i* is also used in notation

Forward process



Forward process

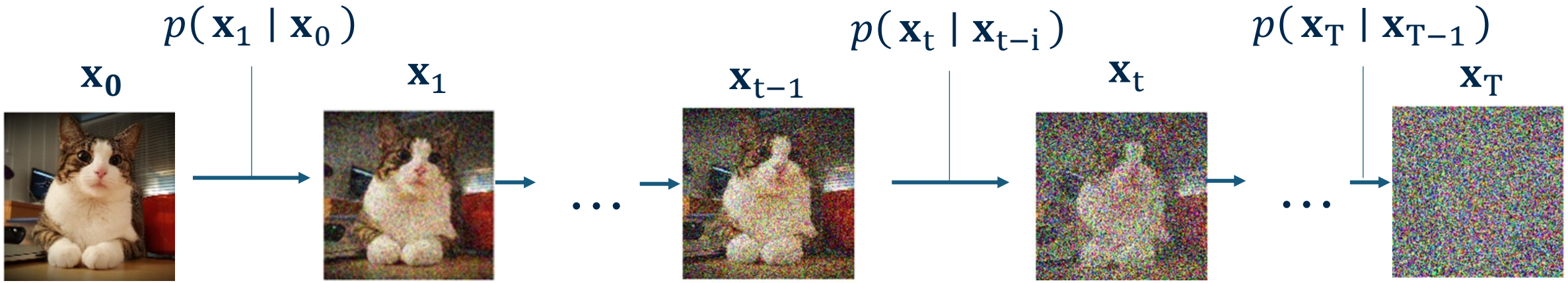


$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) := \mathcal{N}\left(\mathbf{x}_t; \sqrt{1-\beta_t^2}\mathbf{x}_{t-1}, \beta_t^2\mathbf{I}\right)$$

Gaussian perturbation step

Mean **Variance (depends only on β_t^2 !)**

Forward process



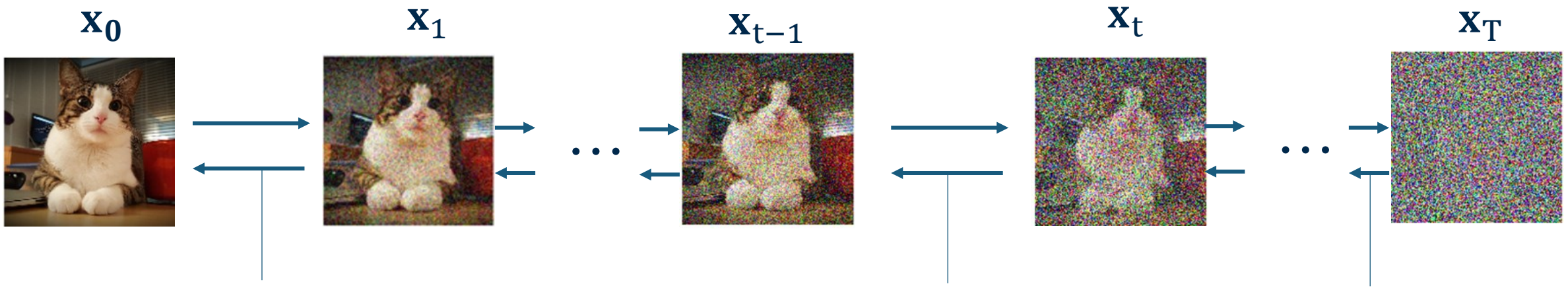
This is mathematical equivalent to sampling as follows

$$\mathbf{x}_t = \alpha_t \mathbf{x}_{t-1} + \beta_t \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ i.i.d.}$$

This is the same reparametrization trick from VAE!

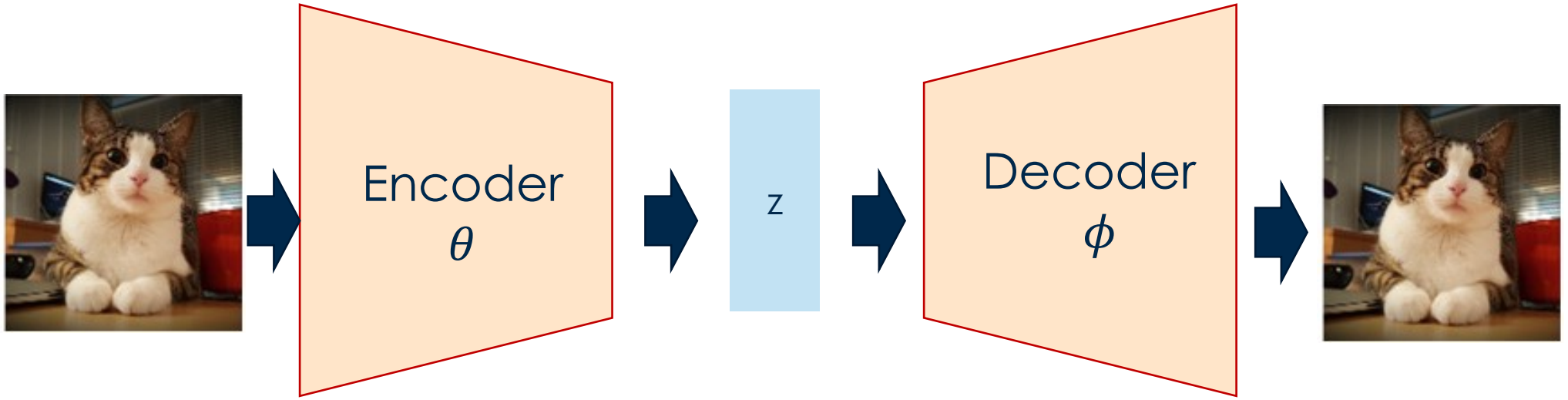
Reverse process

Parameterized (learnable) function with weights ϕ learns to denoise



Our **goal** is to learn this function

DDPM from a variational perspective



VAE: trainable encoder and decoder

DDPM: fixed encoder - trainable decoder – hierarchical process

The forward process: sampling at time t

The noise at the time T can be computed by applying the chain:

$$p(\mathbf{x_{1:T}} \mid \mathbf{x_0}) = \prod_{t=1}^{T} p(\mathbf{x_t} \mid \mathbf{x_{t-1}})$$

If we assume a Gaussian transition kernel

$$p(\mathbf{x_t} \mid \mathbf{x_{t-1}}) \coloneqq \mathcal{N}\left(\mathbf{x_t}; \sqrt{1-\beta_t^2}\mathbf{x_{t-1}}, \beta_t^2\mathbf{I}\right)$$

we can compute a closed form:

$$p_t(\mathbf{x_t} \mid \mathbf{x_0}) = \mathcal{N}\left(\mathbf{x_t}; \overline{\alpha_t}, \mathbf{x_0}, \left(1-\overline{\alpha_t^2}\right)\mathbf{I}\right)$$

with $\bar{\alpha}_t \coloneqq \prod_{k=1}^{t} \sqrt{1-\beta_k^2} = \prod_{k=1}^{t} \alpha_k$

Step-by-step recursion

$$\mathbb{E}[\mathbf{x_1}|\mathbf{x_0}] = \alpha_1 \mathbf{x_0} + \sigma_1 \underbrace{\mathbb{E}[\boldsymbol{\epsilon_1}]}_{=\mathbf{0}} = \alpha_1 \mathbf{x_0}$$

$$\mathbb{E}[\mathbf{x_2}|\mathbf{x_0}] = \alpha_2, \mathbb{E}[\mathbf{x_1}|\mathbf{x_0}] + \sigma_2 \underbrace{\mathbb{E}[\boldsymbol{\epsilon_2}]}_{=\mathbf{0}} = \alpha_2 \alpha_1 \mathbf{x_0}$$

$$\mathbb{E}[\mathbf{x_3}|\mathbf{x_0}] = \alpha_3, \mathbb{E}[\mathbf{x_2}|\mathbf{x_0}] = \alpha_3 \alpha_2 \alpha_1 \mathbf{x_0}$$

Recall: the expected value of a random variable is its **weighted average**, where each possible value is weighted by its probability of occurring. It represents the "center of mass" of the probability distribution. For a Gaussian distribution, it corresponds to the mean

$$\mathbb{E}[\mathbf{x_t}|\mathbf{x_0}] = \overline{\alpha_t}, \mathbf{x_0}, \qquad \overline{\alpha_t} := \prod_{k=1}^{t} \alpha_k$$

Sampling at time t

The closed form

$$p_t(\mathbf{x}_t \mid \mathbf{x_0}) = \mathcal{N}\left(\mathbf{x}_t; \overline{\alpha_t}, \mathbf{x_0}, \left(1 - \overline{\alpha_t^2}\right)\mathbf{I}\right)$$

can be rewritten as

$$\mathbf{x}_t = \overline{\alpha_t}\mathbf{x_0} + \sqrt{1 - \overline{\alpha_t^2}}\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Take home point #1

We do not need to iteratively add noise, we can directly sample the noisy image at each step of the forward process

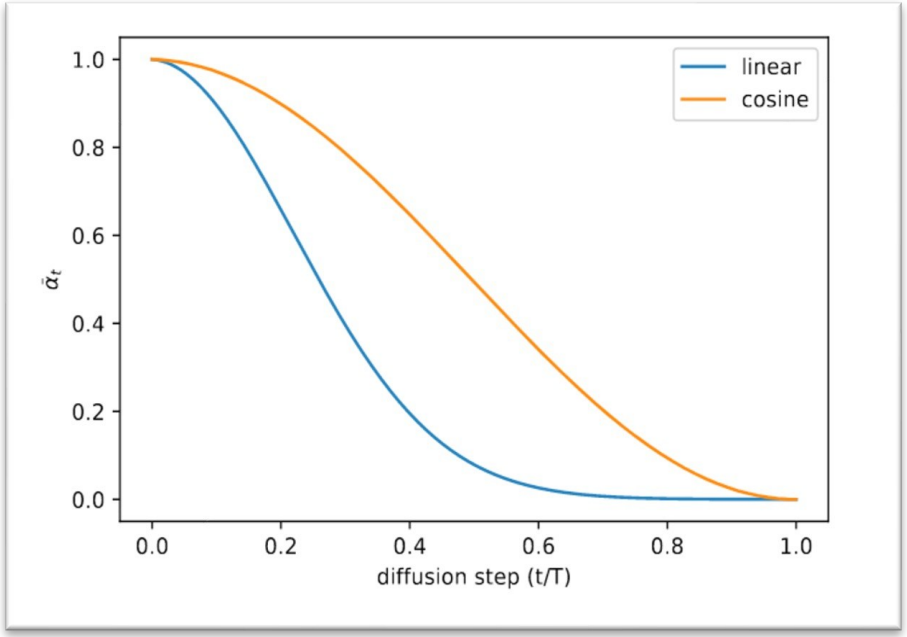
Noise schedule

Linear schedule

- Classic DDPM default (Ho et al., 2020).
- The image is visually recognisable at low t; by t ≈ 0.5–0.7 most signal is gone.
- *Problem: destroys signal too fast — many steps wasted in the 'all noise' regime.*

Cosine Schedule

- Proposed by Nichol & Dhariwal (2021) for improved quality.
- Keeps more semantic structure at mid-noise levels → better gradient signal.
- *Key intuition: visual content degrades gradually; the model can learn fine details at low noise.*



Sampling at time t

The closed form

$$p_t(\mathbf{x_t} \mid \mathbf{x_0}) = \mathcal{N}\left(\mathbf{x_t}; \overline{\alpha_t}\mathbf{x_0}, \left(1 - \overline{\alpha_t^2}\right)\mathbf{I}\right)$$

can be rewritten as

$$\mathbf{x}_t = \overline{\alpha_t}\mathbf{x_0} + \sqrt{1 - \overline{\alpha_t^2}}\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Take home point #2

The sequence $\{\beta_t\}$ is a pre-determined, monotonically increasing noise schedule, $\beta_t \in (0,1)$. As $T \to \infty$ and the noise schedule increases:

$$p_T(\mathbf{x}_T \mid \mathbf{x_0}) \longrightarrow \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad \text{as } T \to \infty$$

The reverse process

Our goal is to approximate the true (unknown) reverse transition $\mathbf{q}$

$$\mathbb{E}_{\mathbf{p_t(x_t)}}\left[D_{\mathrm{KL}}\left(\mathrm{q}(\mathbf{x_{t-1}} \mid \mathbf{x_t}) \| p_{\phi}(\mathbf{x_{t-1}} \mid \mathbf{x_t})\right)\right]$$

This function is intractable, but we can make it tractable by conditioning on a clean data sample $\mathbf{x_0}$.

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0}) = \frac{q(\mathbf{x_t} \mid \mathbf{x_{t-1}}) q(\mathbf{x_{t-1}} \mid \mathbf{x_0})}{q(\mathbf{x_t} \mid \mathbf{x_0})}$$

This formulation is tractable and admits a closed form analytical solution

Modeling the reverse transition

Assumption: the reverse transition is Gaussian

$$p_\phi(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}\left(\mathbf{x}_{t-1};\ \mu_\phi(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\right)$$

Mean

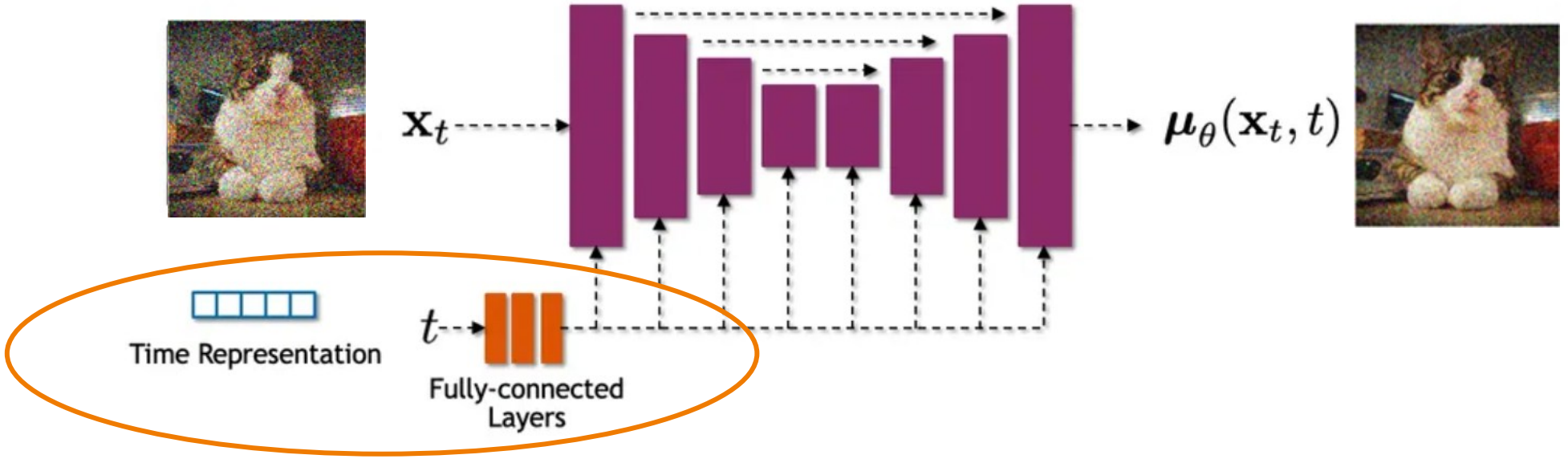
- Parametric function $\mu_\phi(\cdot, t): \mathbb{R}^D \to \mathbb{R}^D$
- Computes the "mean" image
- Takes as input the previous image and the time step

Variance

- fixed
- function of $\bar{\alpha}_t$ and β_t

Computing the reverse transition

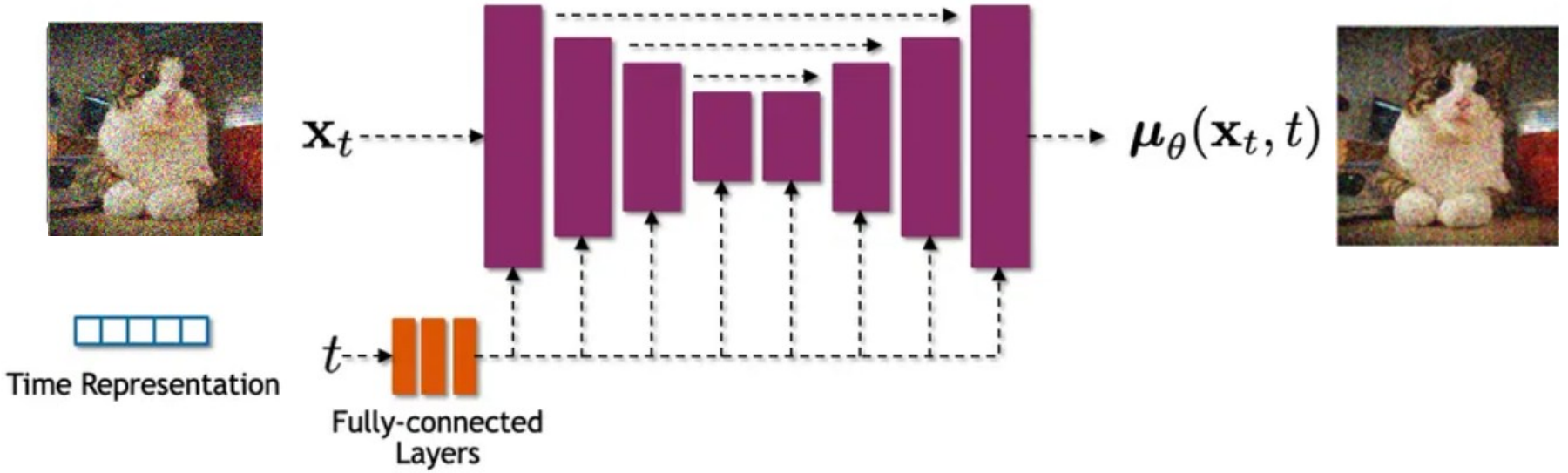
Diffusion model often use U-net architectures with residual blocks, skip-connections and self-attention layers.



The time step is given as input (using, e.g., sinusoidal encoding) → the reverse function depends on the step!

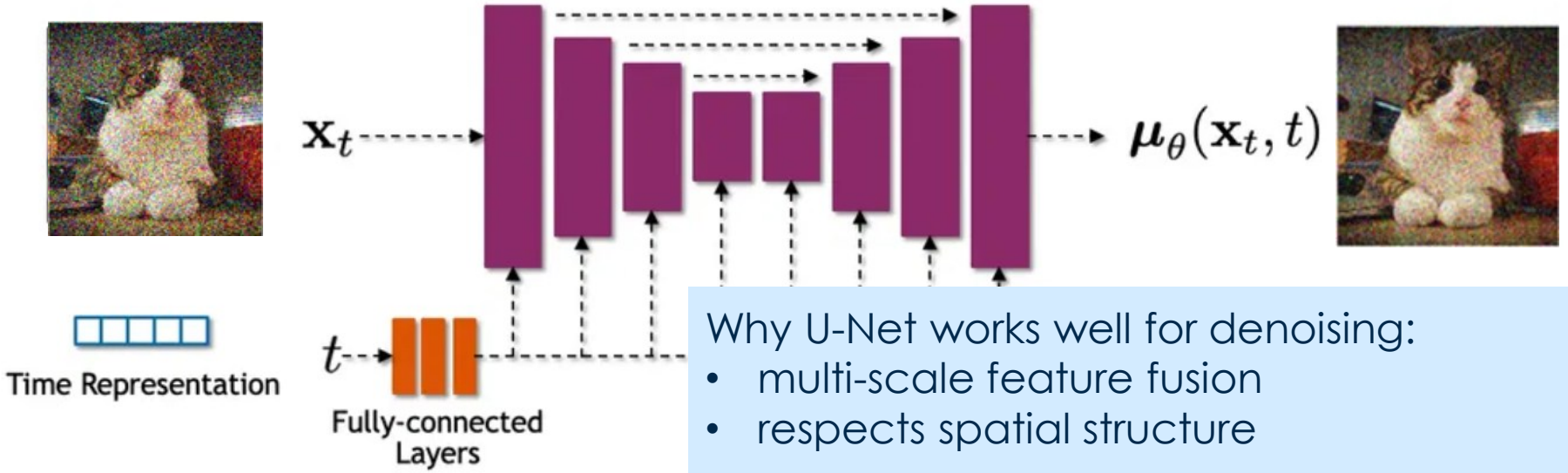
Computing the reverse transition

Diffusion model often use U-net architectures with residual blocks, skip-connections and self-attention layers.



Computing the reverse transition

Diffusion model often use U-net architectures with residual blocks, skip-connections and self-attention layers.



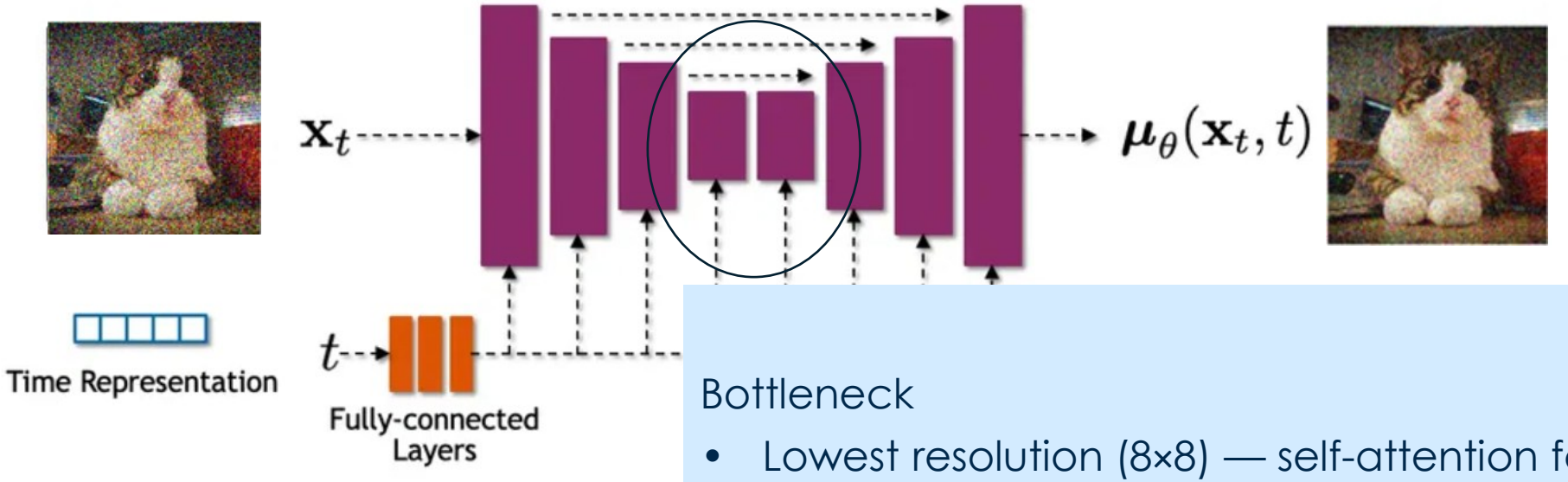
Why U-Net works well for denoising:

- multi-scale feature fusion
- respects spatial structure

Other architectures (e.g. Transformer based) are possible

Computing the reverse transition

Diffusion model often use U-net architectures with residual blocks, skip-connections and self-attention layers.



Bottleneck

- Lowest resolution (8×8) — self-attention for global coherence.

Training the reverse transition

Recall: the reverse transition is Gaussian $p_\phi(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1};\ \mu_\phi(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$

Given a training sample $\mathbf{x_0}$, we want

$$p_\phi(\mathbf{x_{t-1}} \mid \mathbf{x_t})$$

to be as close as possible to

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0})$$

Intuition: if the learned reverse process is supposed to subtract away the noise, then whenever we're working with a specific $\mathbf{x_0}$ it should subtract it away exactly as exact reverse process would have.

Learn to approximate the mean $\mu_\phi(\mathbf{x_t}, t)$ from $\mu_q(\mathbf{x_t}, \mathbf{x_0})$

$$\mathcal{L}_{\text{diffusion}}(\mathbf{x_0}; \phi) = \sum_{t=1}^{T} \frac{1}{2\sigma_t^2} \left\| \mu_\phi(\mathbf{x_t}, t) - \mu_q(\mathbf{x_t}, \mathbf{x_0}, t) \right\|_2^2 + C$$

Averaging over the data distribution and omitting C, the final DDPM training objective is:

$$\mathcal{L}_{\text{DDPM}}(\phi) = \sum_{t=1}^{T} \frac{1}{2\sigma_t^2} \mathbb{E}_{\mathbf{x_0}} \mathbb{E}_{p(\mathbf{x_t} \mid \mathbf{x_0})} \left[\left\| \mu_\phi(\mathbf{x_t}, t) - \mu_q(\mathbf{x_t}, \mathbf{x_0}, t) \right\|_2^2 \right]$$

Training the reverse transition

Recall: the reverse transition is Gaussian $p_\phi(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}\left(\mathbf{x}_{t-1};\ \mu_\phi(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\right)$

Given a training sample $\mathbf{x_0}$, we want

$$p_\phi(\mathbf{x_{t-1}} \mid \mathbf{x_t})$$

to be as close as possible to

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0})$$

Intuition: if the learned reverse process is supposed to subtract away the noise, then whenever we're working with a specific $\mathbf{x_0}$ it should subtract it away exactly as exact reverse process would have.

In practice, either:

$$\|\mu_\phi(\mathbf{x}_t, t) - \mu_q(\mathbf{x}_t, \mathbf{x}_0, t)\|_2^2 \propto \|\mathbf{x}_\phi(\mathbf{x}_t, t) - \mathbf{x_0}\|_2^2, \quad \mathbf{x_0} \sim p_{\text{data}}$$

$\text{Unet}_\phi(\mathbf{x}_t, t) = \mathbf{x}_\phi(\mathbf{x}_t, t)$ predicts denoised image at step t

Training the reverse transition

Recall: the reverse transition is Gaussian $p_\phi(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1};\ \mu_\phi(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$

Given a training sample $\mathbf{x_0}$, we want

$$p_\phi(\mathbf{x_{t-1}} \mid \mathbf{x_t})$$

to be as close as possible to

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0})$$

Intuition: if the learned reverse process is supposed to subtract away the noise, then whenever we're working with a specific $\mathbf{x_0}$ it should subtract it away exactly as exact reverse process would have.

In practice, either:

$$\|\mu_\phi(\mathbf{x_t}, t) - \mu_q(\mathbf{x_t}, \mathbf{x_0}, t)\|_2^2 \propto \|\mathbf{x}_\phi(\mathbf{x_t}, t) - \mathbf{x_0}\|_2^2, \quad \mathbf{x_0} \sim p_{\text{data}}$$

$\text{Unet}_\phi(\mathbf{x_t}, t) = \mathbf{x}_\phi(\mathbf{x_t}, t)$ predicts denoised image at step t

or:

$$\|\mu_\phi(\mathbf{x_t}, t) - \mu_q(\mathbf{x_t}, \mathbf{x_0}, t)\|_2^2 \propto \|\boldsymbol{\epsilon}_\phi(\mathbf{x_t}, t) - \boldsymbol{\epsilon}\|_2^2, \quad \boldsymbol{\epsilon} \sim p_{\text{noise}}$$

$\text{Unet}_\phi(\mathbf{x_t}, t) = \boldsymbol{\epsilon}_\phi(\mathbf{x_t}, t)$ predicts **noise** added during training

Training the reverse transition

Recall: the reverse transition is Gaussian $p_\phi(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1};\ \mu_\phi(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$

Given a training sample $\mathbf{x_0}$, we want

$$p_\phi(\mathbf{x_{t-1}} \mid \mathbf{x_t})$$

to be as close as possible to

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0})$$

Intuition: if the learned reverse process is supposed to subtract away the noise, then whenever we're working with a specific $\mathbf{x_0}$ it should subtract it away exactly as exact reverse process would have.

In practice, either:

$$\|\mu_\phi(\mathbf{x}_t, t) - \mu_q(\mathbf{x}_t, \mathbf{x}_0, t)\|_2^2 \propto \|\mathbf{x}_\phi(\mathbf{x}_t, t) - \mathbf{x_0}\|_2^2, \quad \mathbf{x_0} \sim p_{\text{data}}$$

$\text{Unet}_\phi(\mathbf{x}_t, t) = \mathbf{x}_\phi(\mathbf{x}_t, t)$ predicts denoised image at step t

or:

$$\|\mu_\phi(\mathbf{x}_t, t) - \mu_q(\mathbf{x}_t, \mathbf{x}_0, t)\|_2^2 \propto \|\boldsymbol{\epsilon}_\phi(\mathbf{x}_t, t) - \boldsymbol{\epsilon}\|_2^2, \quad \boldsymbol{\epsilon} \sim p_{\text{noise}}$$

$\text{Unet}_\phi(\mathbf{x}_t, t) = \boldsymbol{\epsilon}_\phi(\mathbf{x}_t, t)$ predicts **noise** added during training → **empirically the best option**

Training loop in practice

For each batch – until convergence

1. Sample noise to add to each image in the batch
2. Sample a random timestep for each image t
3. Add noise to the clean images according to the noise magnitude
4. Predict the noise residual
5. Compute the MSE loss
6. Update the gradients

No need to compute the entire chain for each image – we can predict the noisy image at an arbitrary time step

Algorithm 1 Training

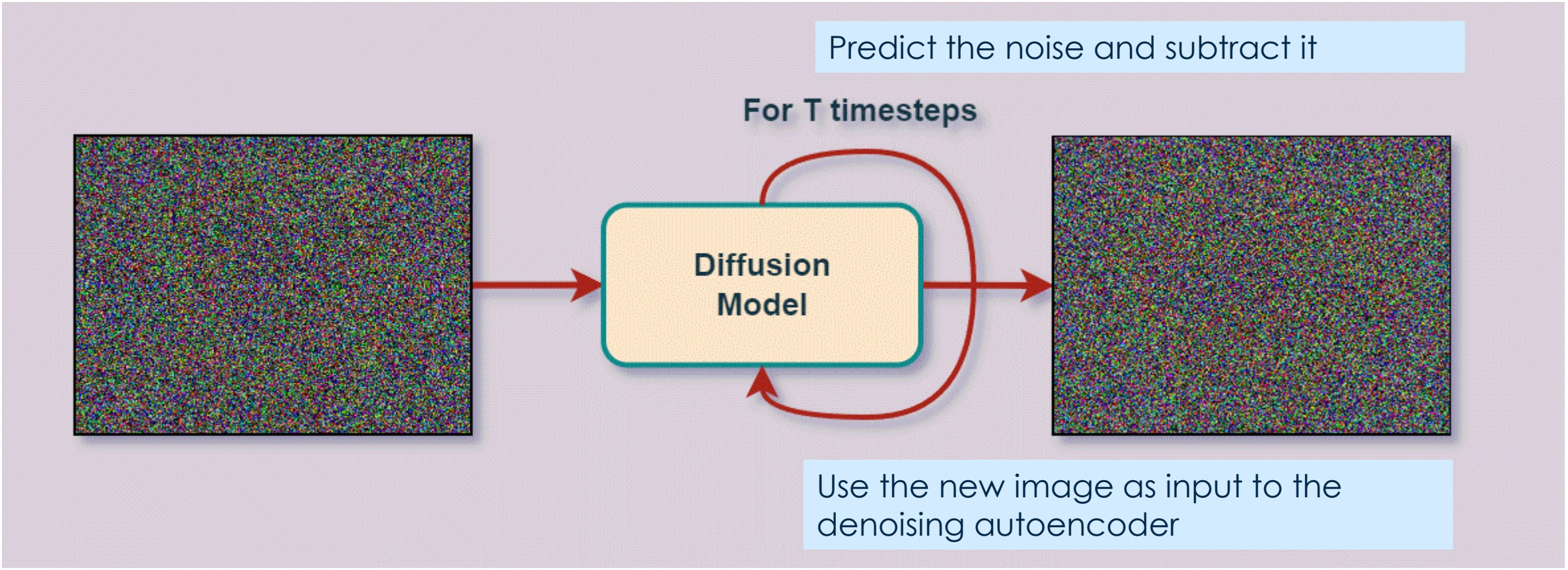
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Image source: Ho et al. 2020

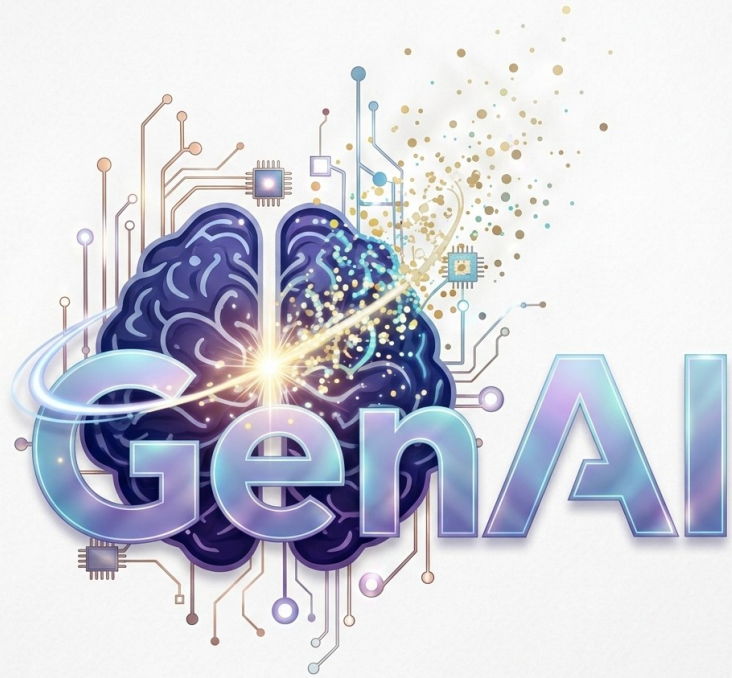
At inference time



Source: https://learnopencv.com

Politecnico di Torino

But why latent diffusion models?



Problems with pixel-space diffusion

How much does it cost to sample an image $1024 \times 1024 \times 1000$ timesteps?

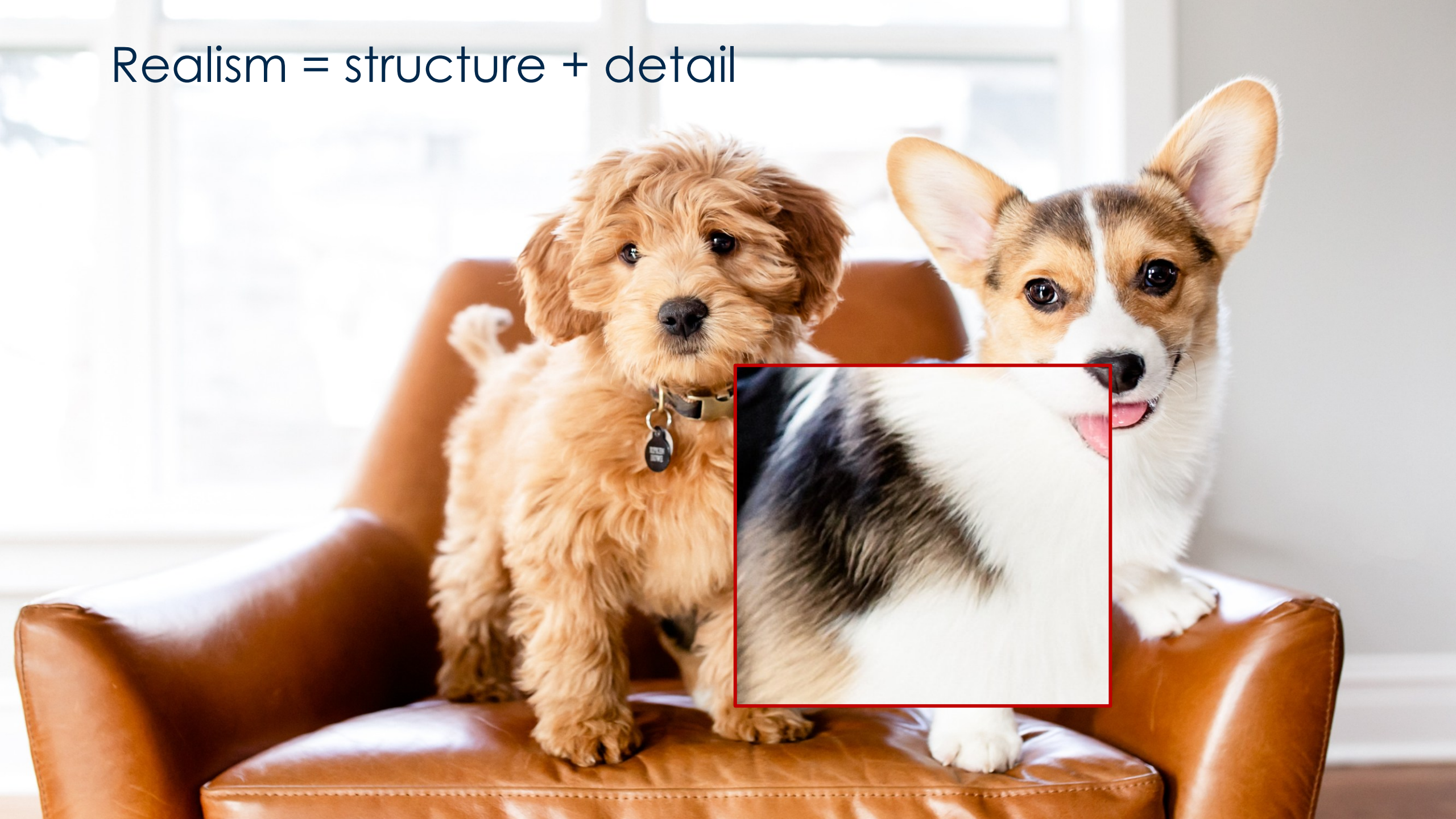
Problems with pixel-space diffusion

How much does it cost to sample an image 1024 × 1024 × 1000 timesteps?

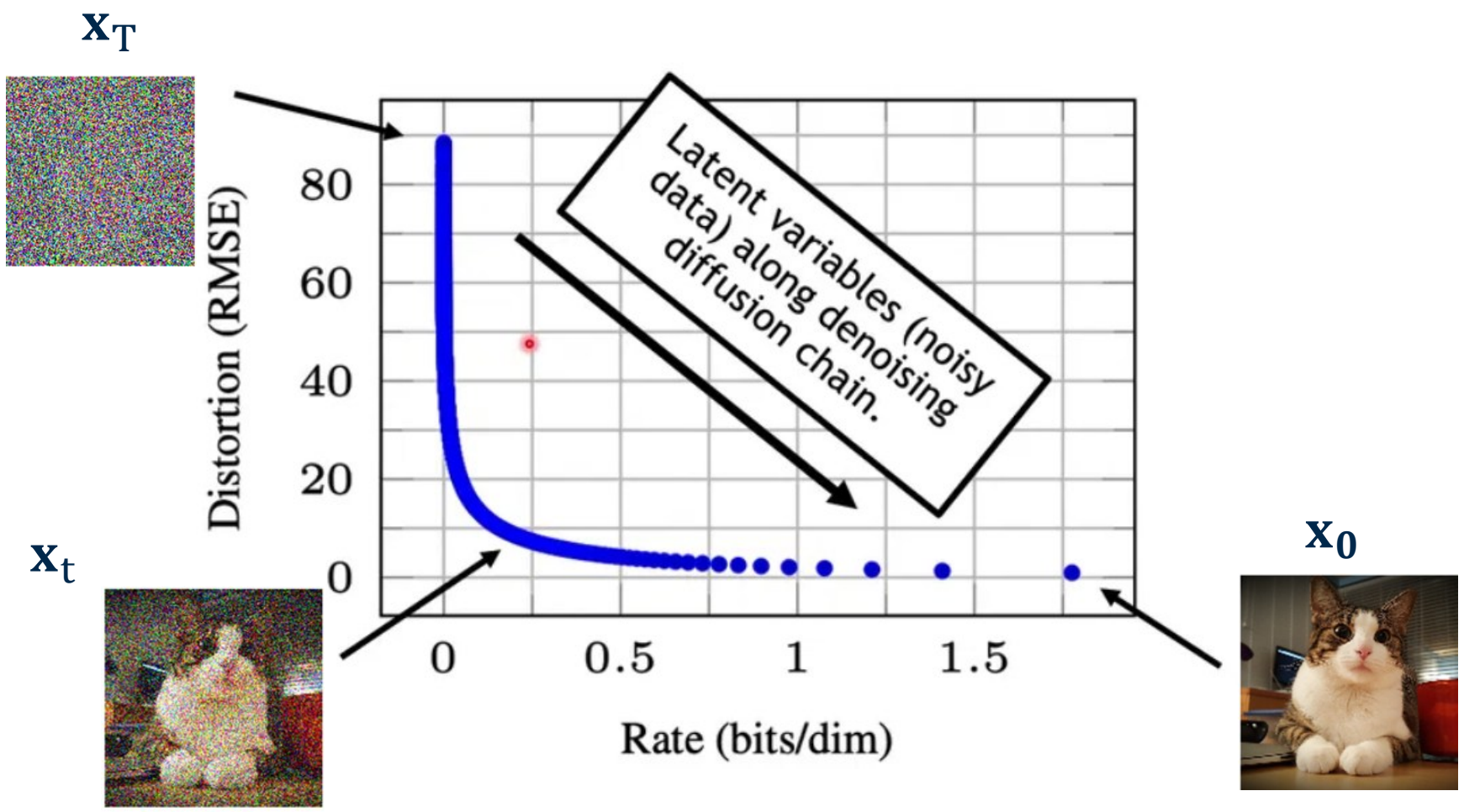
- Computational cost **scales with resolution** $\mathcal{O}(\mathrm{H} \times \mathrm{W})$ per each timestep t
- DDPM originally worked at 32×32 or 64×64 — scaling to high resolution is extremely expensive.
- Each denoising step requires a **full forward pass** of a large U-Net over the full spatial resolution.
- 1000 sequential steps: real-time generation at high-res is infeasible on a single GPU

Realism = structure

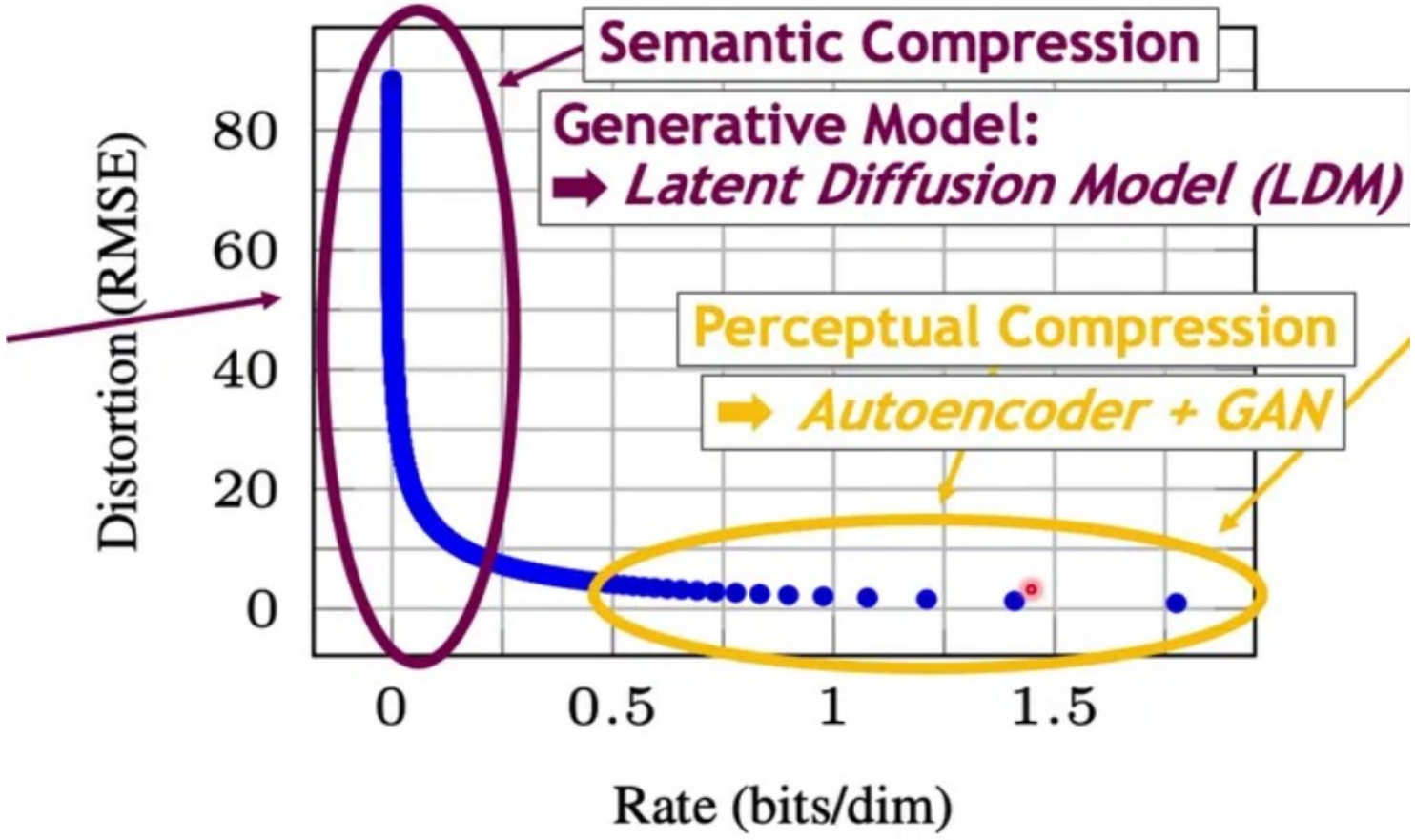
Realism = structure + detail



Semantic compression

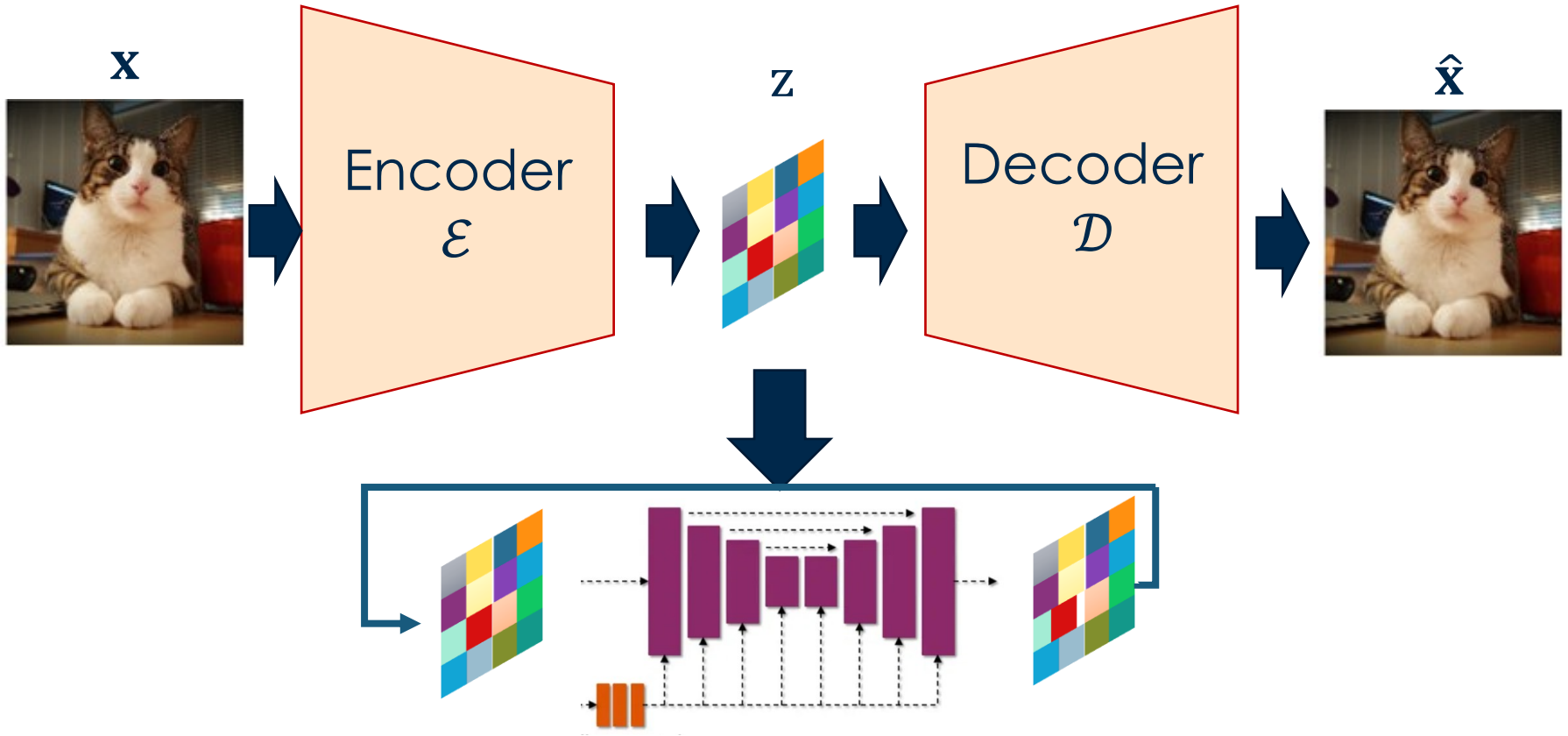


Idea: decouple semantics from detail



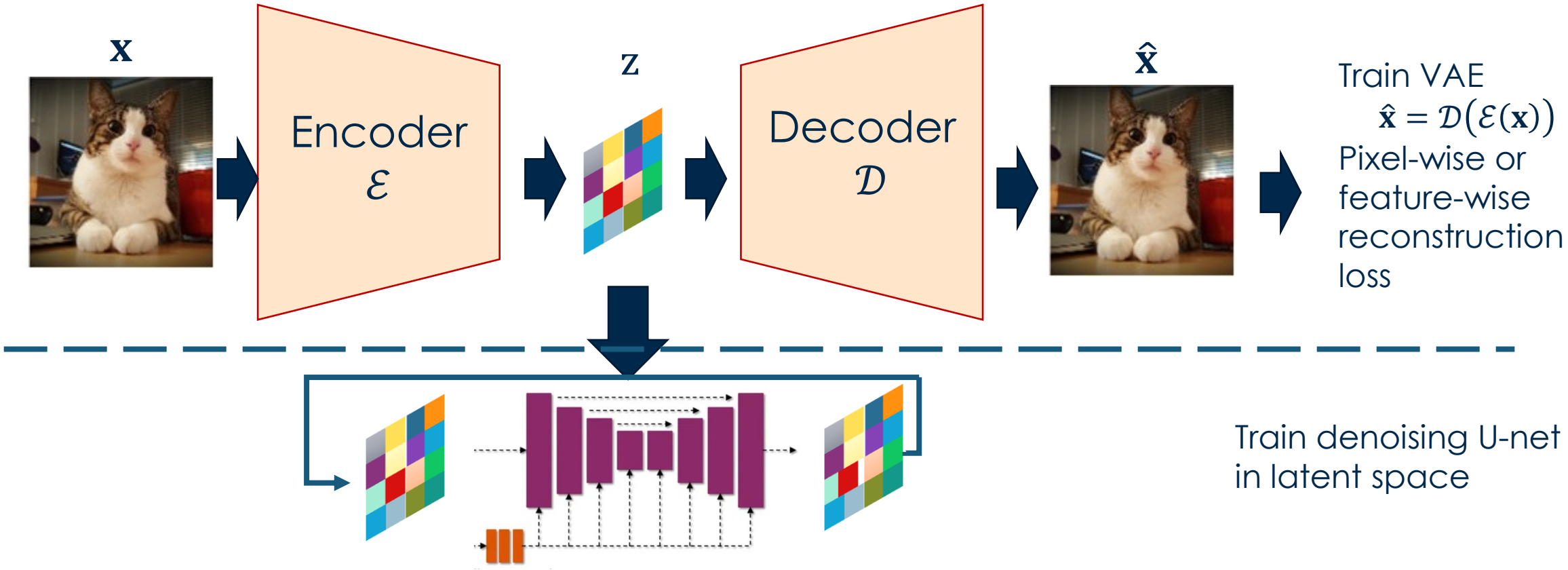
Latent diffusion models

The LDM idea (Rombach et al., 2022): first compress image to latent z with a perceptual VAE, then run diffusion entirely in latent space



Latent diffusion models

The LDM idea (Rombach et al., 2022): first compress image to latent z with a perceptual VAE, then run diffusion entirely in latent space



From pixel to latent space

- Two-stage training: original model
  - Stage 1 — Train a perceptual autoencoder to compress images to z and reconstruct faithfully.
  - Stage 2 — Train a DDPM entirely in the low-dimensional latent space z.
- One-stage training: possible in more recent models
- At inference: sample $\tilde{\mathbf{z}}$ the diffusion model, then decode $\tilde{\mathbf{z}} \rightarrow \mathbf{x}$ via $\mathcal{D}$.
- The two stages are decoupled — the diffusion model never sees raw pixels.

Reduce computational load: 64×64 latent vs. 512×512 pixel → 64× fewer operations per diffusion step!

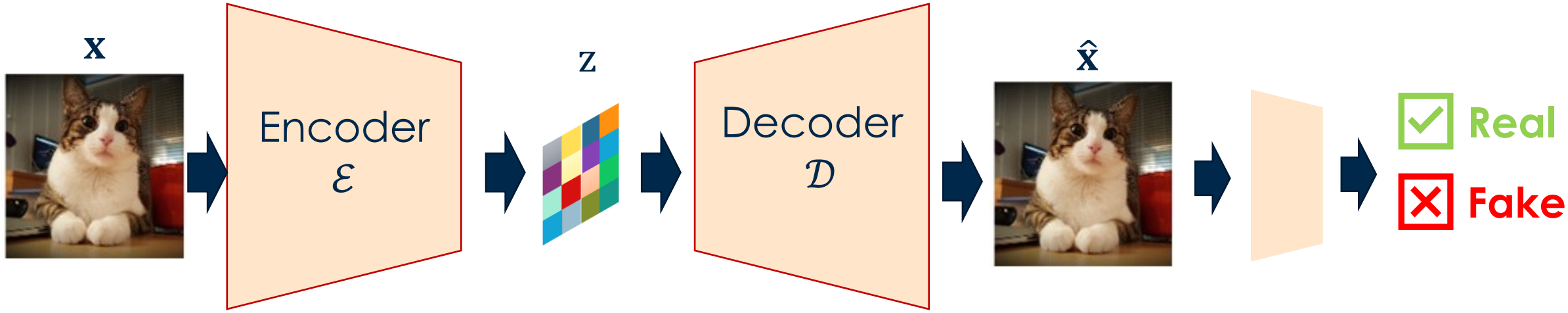
From pixel to latent space

- Two-stage training: original model
  - Stage 1 — Train a perceptual autoencoder to compress images to z and reconstruct faithfully.
  - Stage 2 — Train a DDPM entirely in the low-dimensional latent space z.
- One-stage training: possible in more recent models
- At inference: sample $\tilde{\mathbf{z}}$ the diffusion model, then decode $\tilde{\mathbf{z}} \rightarrow \mathbf{x}$ via $\mathcal{D}$.
- The two stages are decoupled — the diffusion model never sees raw pixels.

Flexible method: change domain by changing the VAE block

The VAE component

Adversarial discriminator (improve fidelity)

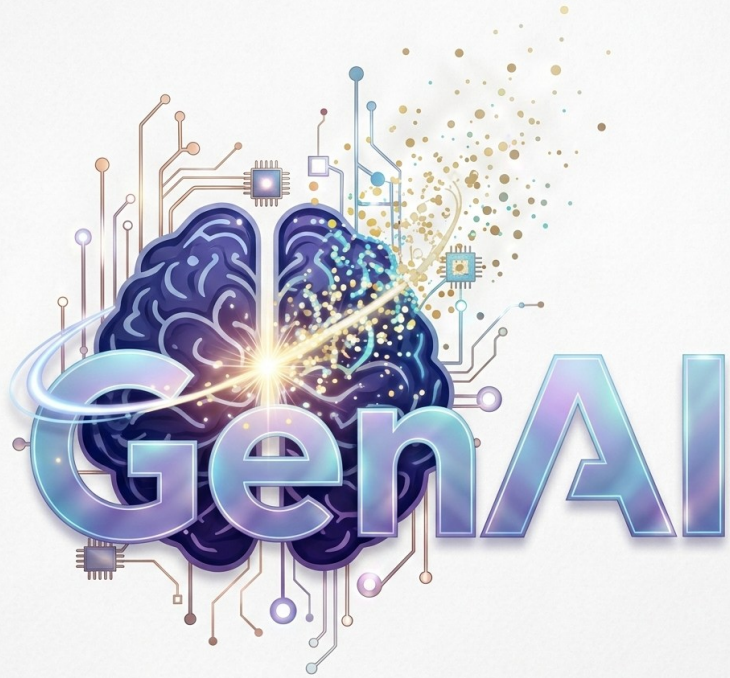


Regularize latent space

1) Use KL-divergence w.r.t. to prior (as in regular VAE)
2) Discretize encoding as in VQ-VAE

Politecnico di Torino

Putting everything together



Sunset in a valley with trees and mountains

A decadent chocolate cake adorned with decorative sugar art

An emu wearing sun glasses and chilling on a beach

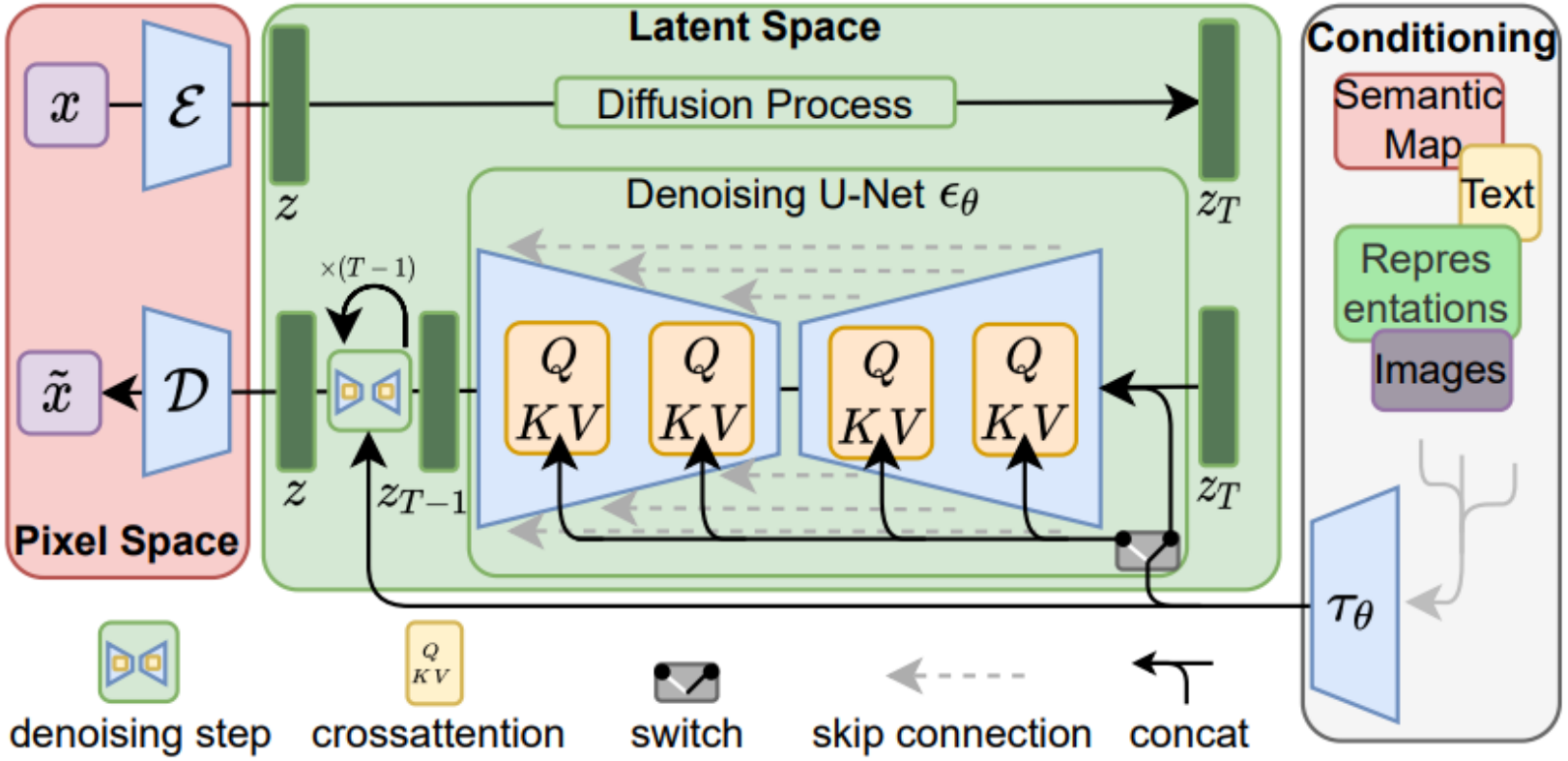
An old man with green eyes and a beard

A beaver dressed in a vest, wearing glasses and a vibrant necktie, in a library

A painting of an adorable rabbit sitting on a colourful splash

A traditional tea house in a tranquil garden with blooming cherry blossom trees

Latent diffusion model



Rombach et al. "High-resolution image synthesis with latent diffusion models." *CVPR* 2022.

Towards conditional synthesis

Unconditional synthesis

$$\mathcal{L}_{\mathrm{LDM}} = \mathbb{E}_{\mathcal{E}(\mathrm{x}),\epsilon,\mathrm{t}}\left[\left\|\boldsymbol{\epsilon}_{\phi}(\boldsymbol{z}_{\mathrm{t}}, \mathrm{t}) - \boldsymbol{\epsilon}\right\|_2^2\right]$$

$$\mathcal{L}_{\mathrm{LDM}} = \mathbb{E}_{\mathcal{E}(\mathrm{x}),\epsilon,\mathrm{y},\mathrm{t}}\left[\left\|\boldsymbol{\epsilon}_{\phi}(\boldsymbol{z}_{\mathrm{t}}, \tau_{\theta}(\mathrm{y}), \mathrm{t}) - \boldsymbol{\epsilon}\right\|_2^2\right]$$

Conditional synthesis

Domain-specific expert
Co-trained with the LDM

Conditional generation

Cross-Attention Mechanism (For Text/Class Inputs): the text is converted into embeddings (vectors) via a language model (like CLIP) and injected into the denoising U-Net through cross-attention layers.

Concatenation Mechanism (For Spatial Inputs): When the input is spatially aligned—such as image-to-image translation, inpainting, or semantic maps—the switch directs the input to be concatenated directly to the noisy latent input of the U-Net

Semantic synthesis

Training at scale: Exponential Moving Average

Latent diffusion models are heavy models:

- Small batch sizes create instabilities during training

During training, two copies of the U-Net are maintained in parallel:

- θ (optimizer weights): updated at every gradient step via Adam — noisy, can fluctuate.
- θ_{EMA} (EMA weights): updated as a running average — smooth, stable trajectory through parameter space.

$$\mathrm{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \mathrm{EMA}_{t-1}, \quad \alpha \in (0,1)$$

Why diffusion models won

Sample quality

- Progressive denoising yields sharp, high-fidelity images.
- Early steps determine global structure; later steps add fine detail.

Diversity

- Stochastic reverse process explores the full data distribution.
- No mode collapse unlike GANs.

Training stability

- Fixed encoder — only the denoising decoder is learned.
- Well-conditioned denoising subproblems at each noise level.

Major generative model families

Autoregressive Models

GPT, PixelCNN, WaveNet

Computes exact likelihood
+ Multi-modal, flexible model
- Slow inference (token-by-token)
- Depends on tokenization

Variational Autoencoders

VAE, β-VAE, VQVAE

Explicit latent space modelling
May serve as building blocks for other models
+ Interpretability of the feature space
+ Fast inference
+ Effective representation learning
- Blurry images, low realism

GANs

GAN, StyleGAN, BigGAN

Implicit distribution
+ Fast inference (single step)
+ Photo-realism
- Unstable training

Normalizing Flows

Glow, RealNVP, MAF

Exact inference

Diffusion Models

DDPM, Stable Diffusion, DALL-E

State of the art
Denoising in latent space
+ High quality image synthesis
+ Flexible and highly controllable synthesis
- Slow inference
- Requires massive pretraining

Energy-Based Models

EBM, NCSN, ScoreNet

Flexible density

Bibliography

https://the-principles-of-diffusion-models.github.io/

See chapters 1, 2, 8, 9

Neurips 2023 Tutorial Latent diffusion models: is the generative AI revolution happening in latent space?

Denoising diffusion probabilistic models https://arxiv.org/abs/2006.11239

High-Resolution Image Synthesis with Latent Diffusion Models https://arxiv.org/abs/2112.10752

Emu: Enhancing Image Generation Models Using Photogenic Needles in a Haystack https://arxiv.org/abs/2309.15807

Classifier-Free Diffusion Guidance https://arxiv.org/abs/2207.12598

Adding Conditional Control to Text-to-Image Diffusion Models https://arxiv.org/abs/2302.05543